Collections

1. Java Collection (The Container) គិតវាដូចជា កន្លែងផ្ទុក ធាតុចូល ដែលវាអាច ដាក់ទូរ multiple elements into a single unit.

- In Java, a Collection is essentially a specialized container—an object that groups multiple elements into a single unit.

ដាក់ចឹងដើម្បីស្រួលធ្វើការ រក្សារទុក, ទាញវាមកវិញ និងធ្វើការកែប្រែ សំខាន់គឺយើងមិនបាច់ខ្វល់ទៅលើទំហំវានោះទេ

Ex: បើយើងប្រើ Array យើងត្រូវបង្កើត Size វាមុន ចឹងបើវាត្រូវការបន្ថែមទៀត វានឹងពិបាកធ្វើការកែប្រែ តែបើយើងបង្កើតវាក្នុង Container then put the element inside that container it can manage.

1.1 The Collection Interface and the Collections Framework.

- The Interface: A single "blueprint" that defines the basic operations (adding, removing, clearing) for most data structures in Java.

- The Framework: A unified architecture that includes interfaces, implementations (classes like ArrayList), and algorithms (like sorting or searching).

1-Interface (The "Blueprint")

វាជាកុងត្រាដែលប្រាប់ទៅ class ដែល implement វាថាត្រូវធ្វើអ្វីខ្លះ សម្រាប់ class នឹង។ តែត្រូវចាំ Interface នឹងវាមិនមែនជាអ្នកប្រាប់ ពីរបៀបធ្វើនោះទេ

In Java, an Interface is a contract. It tells a class what to do, but not how to do it.

មានន័យថា បើវាយក collection មក Implement វាត្រូវតែអាចប្រើប្រាស់ Method ក្នុងនឹងបាន

```
Author:          Josh Bloch, Neal Gafter
Type parameters: <E> – the type of elements in this collection

public interface Collection<E> extends Iterable<E> {
    // Query Operations

    Returns the number of elements in this collection. If this collection contains more than
    Integer.MAX_VALUE elements, returns Integer.MAX_VALUE.

    Returns: the number of elements in this collection

    @Contract(pure = true)
    int size();

    Returns true if this collection contains no elements.

    Returns: true if this collection contains no elements

    boolean isEmpty();

    Returns true if this collection contains the specified element. More formally, returns
    true if and only if this collection contains at least one element e such that Objects.
```

1. The Interface: The "Blueprint"

In Java, an **Interface** is a contract. It tells a class **what** to do, but not **how** to do it.

Real-World Analogy: The Wall Socket

- **The Blueprint:** Every wall socket must have holes for a plug and provide electricity.
- **The Operation:** You know you can `plugIn()` and `getPower()`.
- **The Flexibility:** You don't care if the wires behind the wall are copper or aluminum, or if the power comes from solar or wind. As long as it follows the "Socket Blueprint," your phone charger will work.

In Java (The `Collection` Interface):

The `Collection` interface is a blueprint that says: "If you want to be a Collection, you **must** have these abilities:"

- `add(element)` — Put something inside.
- `remove(element)` — Take something out.
- `size()` — Tell me how many items you have.
- `clear()` — Empty everything.

2-The Framework: The "Complete System"

គេបង្កើត Method ហើយអស់ហើយ យើងចាំយកវាមកប្រើទៅបានហើយ

The Framework is the entire "toolbox." It isn't just the blueprint; it's the factory, the ready-made tools, and the instructions on how to use them efficiently.

The Framework consists of three parts:

A. The Implementations (The Tools)

These are the actual "ready-to-use" classes that follow the blueprints.

- `ArrayList` : A "Smart Array" that grows when it gets full. (Great for fast reading).
- `HashSet` : A "Unique Bag" that prevents duplicates. (Great for checking if an item exists).

B. The Algorithms (The "Auto-Pilot")

These are built-in methods that perform complex tasks on your collections so you don't have to write the logic yourself.

- `Collections.sort(myList)` : Automatically puts your list in order.
- `Collections.reverse(myList)` : Flips the order.
- `Collections.max(myList)` : Finds the biggest number.

Summary Table: Putting it Together

Feature	The Interface (Blueprint)	The Framework (The System)
What is it?	A list of rules/methods.	A library of interfaces + classes + tools.
Analogy	A **Recipe** (Steps to follow).	A **Professional Kitchen** (Recipe + Chefs + Tools).
Example	`List` , `Set` , `Collection`	The `java.util` package.
Purpose	To ensure consistency.	To provide high-performance, ready-made code.

Comparison of Common Methods

All classes that implement the `Collection` interface share these core methods, making the API very consistent:

Method	Description
`add(E e)`	Inserts an element into the collection.
`remove(Object o)`	Removes a specific element.
`contains(Object o)`	Returns `true` if the element exists in the collection.
`size()`	Returns the number of elements.
`isEmpty()`	Checks if the collection has zero elements.
`clear()`	Removes everything from the collection.

Which one should you use?

- Use `ArrayList` if you need to access elements frequently by their position (index).
- Use `LinkedList` if you are constantly adding or removing elements from the middle of the list.
- Use `HashSet` if you just need a "bag" of unique items and don't care about their order.
- Use `TreeSet` if you want your unique items to stay sorted automatically.